

Role-Switching Active Perception for Multi-Arm VLA

最后版思路总结

1. Core Idea

不要把 multi-arm active perception 只看成普通 behavior cloning。更好的表述是：它是一个 role-switching problem。
核心问题是：每个时刻哪条机械臂应该观察，哪条机械臂应该操作，以及什么时候应该切换角色。

```
z_t^i in {0, 1}

0 = perception / observation / information gathering
1 = action / manipulation / world-changing interaction

Z_t = [z_t^0, z_t^1, ..., z_t^{N-1}]
```

2. Role Configuration

对双臂系统，role configuration 可以直接枚举。

```
[0, 0] = both arms observe / prepare
[1, 0] = arm 0 manipulates, arm 1 observes
[0, 1] = arm 0 observes, arm 1 manipulates
[1, 1] = both arms manipulate
```

3. From Modes to Role Stages

给定专家标注的 $Z_0, Z_1, \dots, Z_T$ ，提取 role-switch event：

```
c_t = 1[Z_t != Z_{t-1}]

t=0..20    Z=[0,0]  -> role-stage 0
t=21..55    Z=[1,0]  -> role-stage 1
t=56..80    Z=[0,1]  -> role-stage 2
```

4. Role-Switching Advantage

这里的 advantage 不评价低层 action，而是评价 role transition decision。

给定上一角色 Z_{t-1} 和候选下一角色 Z' ，模型判断现在 stay/switch 到 Z' 是否符合专家的 role-switching 行为。

```
A_switch = 1, if candidate transition matches expert transition
A_switch = 0, otherwise
```

5. Soft Timing Definition

switch boundary 不应该过硬。专家在 $t=50$ 切换，不代表 $t=48$ 或 $t=52$ 一定错。因此可以用一个带时间容忍的 soft advantage。

```
A_sw(t, Z') =
    exp(-|t - tau*| / sigma)
    * (1 - HammingDistance(Z', Z_tau*) / N)

tau*   = nearest expert switch time
Z_tau* = expert role after the switch
N      = number of arms
```

$\sigma = \text{timing tolerance}$

6. Intuition

正确角色加正确时间, advantage 高。正确角色但早一点或晚一点, advantage 中高。错误角色或错误时间, advantage 低。

correct role + correct timing	-> high advantage
correct role + slightly early/late	-> medium-high advantage
correct role + very wrong timing	-> low advantage
wrong role + correct timing	-> low or medium advantage
wrong role + wrong timing	-> low advantage

7. Training Data

从 expert trajectories 里拿到 observation、robot state、language 和 expert role Z_t^* 。然后自动提取 switch points, 并构造 candidate next roles。

```
O_t, q_t, language, Z_t^*
tau_k = {t | Z_t^* != Z_{t-1}^*}
Z' in {0,1}^N
```

7.5 Cumulative Role-Switch Progress

为了更接近 kai_0 , 可以定义一个累计式 role progress, 总进度归一化到 100%。如果专家轨迹有 K 个 role stages, 那么每个 stage 完成贡献 $1/K$ 。

例如有 3 个 role stages, 每个 stage 切换/完成成功贡献约 0.33, 三个 stage 加起来是 1.00。

```
role-stage 0 completed -> +0.33
role-stage 1 completed -> +0.33
role-stage 2 completed -> +0.33
total                  -> 1.00
```

```
P_role(t) in [0, 1]
A_role(t, t') = P_role(t') - P_role(t)
```

7.6 Hybrid with kai_0 -Style Within-Stage Progress

也可以把 kai_0 的 stage 内部 advantage 缝进来。每个 role stage 占总进度的 $1/K$, stage 内部再有一个 0 到 1 的局部进度。

$$P_{\text{role}}(t) = k / K + (1 / K) * p_k(t)$$

k = current role-stage index
 $p_k(t)$ = progress inside role-stage k

$$A_{\text{role}}(t, t') = P_{\text{role}}(t') - P_{\text{role}}(t)$$

7.7 Hybrid Advantage

最后可以把累计 role-stage progress 和 switch correctness 合起来：前者奖励系统沿着专家 role-stage 序列往前走, 后者奖励在正确时间切到正确角色。

```
A_hybrid(t, t')
= [P_role(t') - P_role(t)]
+ beta * A_switch(t', Z')
```

role-stage progress: moving through the sequence
switch advantage: switching at the right time to the right role

8. Negative Candidates

第一版可以非常简单：用随机 corruption 产生负样本。后续再加更难 negative。

```
Z' = random role configuration
Z' = randomly flipped expert role
Z' = wrong target role at a switch point
Z' = switch when expert stays
Z' = stay when expert switches
```

9. Model

训练一个 role-switching advantage

model。输入最近一段历史、上一角色和候选下一角色，输出这个切换是否合理。

```
F_theta(0_{t-H:t}, q_{t-H:t}, language, Z_{t-1}, Z')
-> A_switch
```

10. Ranking Loss

也可以不用 regression，直接做 ranking：专家切换应该比 corrupted 切换分数更高。

```
L_rank =
    -log sigmoid(
        F_theta(history, Z_expert)
        - F_theta(history, Z_corrupt)
    )
```

11. Inference

部署时枚举所有 candidate next roles，选择分数最高的角色。如果最高分不够高，就保持当前角色。

```
for Z' in {0,1}^N:
    score(Z') = F_theta(history, Z_{t-1}, Z')

if max score is high enough:
    switch to argmax_Z' score(Z')
else:
    stay in current role
```

12. Role-Conditioned Policy

选出的当前 role configuration 被送进 VLA/WAM policy，作为 action generation 的条件。

```
pi(a_t | O_t, q_t, language, Z_t)

history
-> role-switching scorer
-> current role configuration
-> role-conditioned VLA / WAM
-> per-arm actions
```

13. Why Switching, Not Just Frame-Wise Mode

逐帧 mode prediction 只问“当前每条臂是什么 mode”。Role switching 问的是“是否应该保持当前角色，还是切换到新的角色配置”。这更接近 active perception 的真实难点。

```
premature switch: manipulating before enough information is gathered
delayed switch: observing for too long without task progress
wrong-arm switch: assigning manipulation to the wrong arm
missed switch: failing to enter a needed cooperative stage
oscillation: switching roles too frequently
```

14. Relationship to kai0

kai0 的 Stage Advantage 问的是：这个 action segment 有没有推进当前 task stage。这里的 Role-Switching Advantage 问的是：这个 role transition 在当前多臂主动感知状态下是否正确。

最像 kai0 的版本是定义 $P_{\text{role}}(t)$ ：每个 role stage 贡献 $1/K$ 的总进度，stage 内部可选地用时间进度或 switch-readiness progress。这样 advantage 就是 $P_{\text{role}}(t') - P_{\text{role}}(t)$ 。

```
kai0:
semantic task stage -> progress advantage -> advantage-conditioned BC

this project:
multi-arm role stage -> switch advantage -> role-conditioned VLA

kai0-style bridge:
 $P_{\text{role}}(t) = k/K + (1/K) * p_k(t)$ 
 $A_{\text{role}}(t, t') = P_{\text{role}}(t') - P_{\text{role}}(t)$ 
```

15. Proposed Contribution

把 multi-arm active perception 表述成 role-switching，而不是隐式 BC。把 per-arm perception/manipulation mode 标注压缩成 temporally coherent role stages。训练一个 role-switching advantage model，根据专家 switch event 的时间接近度和角色结构相似度，为 candidate role transition 打分。最后用这个 role signal condition VLA 或 WAM，让系统显式决定谁看、谁动、什么时候切换。

16. Short Method

最小可行方法可以写成五步。

1. Annotate per-arm perception/action modes.
2. Convert mode sequences into role-switch events.
3. Train a role-switching advantage model with expert switches and corrupted alternatives.
4. At inference, score candidate next roles and decide stay/switch.
5. Feed the selected role into a role-conditioned VLA/WAM policy.

17. Core Claim

Active perception in multi-arm manipulation is fundamentally a role-switching problem. Making role switching explicit should improve generalization across layouts, occlusions, robot roles, and deployment settings.